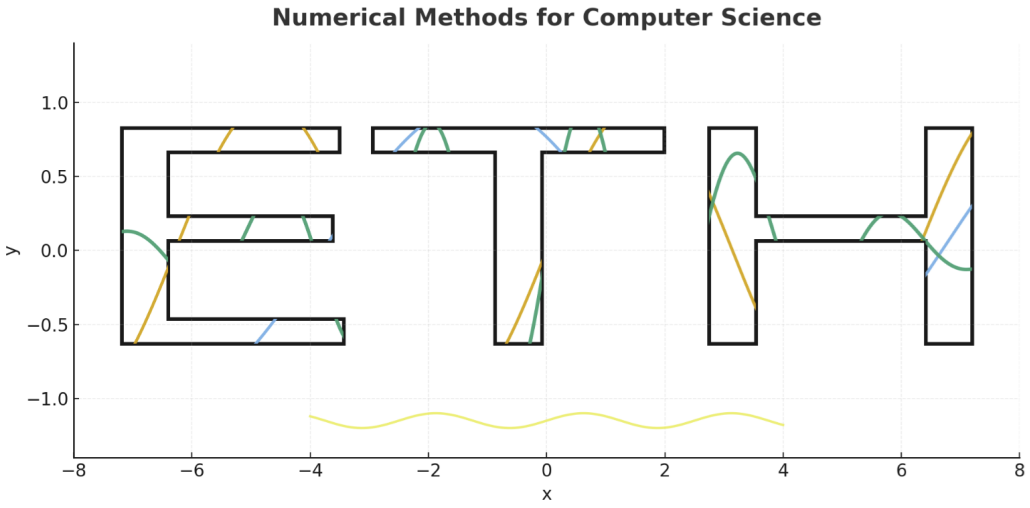

The Discrete Fourier Transform (DFT) and FFT — Week #5

ETH Zurich

DAVID Mihnea-Ștefan

mihdavid@ethz.ch

This week connects continuous Fourier analysis with its discrete counterpart, introducing the DFT and its fast implementation, the FFT. We explore the theory, intuition, and real-world impact in modern signal processing.

Contents

1 Exam? What is relevant

(In-class discussion.)

2 Recap - Discrete Fourier Transform (DFT)

2.1 From continuous to discrete

Up to now we have seen how to expand a *continuous periodic function* into a Fourier series. But in practice we never know $f(t)$ everywhere: we only measure it at discrete sample points

$$t_j = \frac{j}{N}, \qquad j = 0, 1, \ldots, N-1,$$

with values

$$y_j = f(t_j).$$

> Note
>
> **Bridge.** Fourier series answer the question: "If I know the function exactly, what are its frequency coefficients?" The Discrete Fourier Transform (DFT) answers the practical question: "If I only know N sampled values, how do I extract frequency information from them?"

2.2 Trigonometric interpolation viewpoint

Recall: with N samples we can build a trigonometric polynomial of degree at most

$$m = \left\lfloor \frac{N-1}{2} \right\rfloor$$

that interpolates all samples exactly. This interpolant has the form

$$p(t) = \sum_{k=-m}^{m} c_k \, e^{2\pi i k t}.$$

The unknowns are the coefficients c_k, and the equations are the interpolation conditions

$$p(t_j) = y_j, \qquad j = 0, 1, \ldots, N-1.$$

Thus, computing the c_k from the y_j is precisely what the DFT does.

2.3 Roots of unity and orthogonality

The key algebraic building block of the DFT is the *N-th root of unity*:

$$\omega_N = e^{-2\pi i/N}.$$

Step 1: What are they? If we take successive powers of ω_N, we obtain

$$1,\ \omega_N,\ \omega_N^2,\ \ldots,\ \omega_N^{N-1}.$$

These N points lie equally spaced on the unit circle in the complex plane, at angles

$$0,\ -\tfrac{2\pi}{N},\ -\tfrac{4\pi}{N},\ \ldots,\ -\tfrac{2\pi(N-1)}{N}.$$

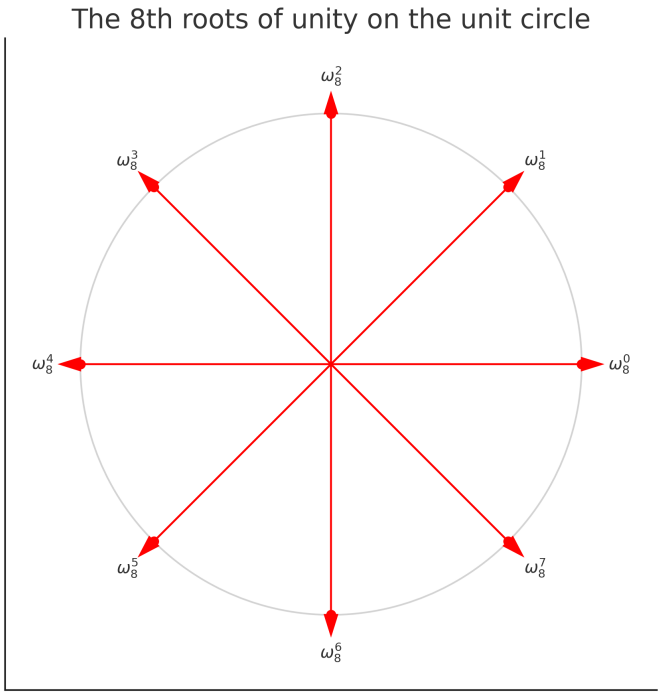

Figure 1: The N-th roots of unity for $N = 8$: equally spaced points on the unit circle, forming a regular octagon. The symmetry explains why their sum is zero (orthogonality).

Step 2: Why are they useful? Imagine drawing arrows (vectors) from the origin to each of these points. Because they are symmetrically placed, the arrows cancel when summed:

$$1 + \omega_N + \omega_N^2 + \cdots + \omega_N^{N-1} = 0.$$

This is true whenever you go once around the circle (i.e. for any nontrivial rotation).

Step 3: The orthogonality relation. More generally, if we take

$$\sum_{j=0}^{N-1} \omega_N^{jk},$$

then

$$\sum_{j=0}^{N-1} \omega_N^{jk} = \begin{cases} N, & k \equiv 0 \pmod{N}, \\ 0, & \text{otherwise.} \end{cases}$$

Note

Intuition. - Think of walking around the unit circle N steps at a time. - If $k = 0$, you never rotate: all vectors are 1, so their sum is N. - If $k \neq 0$, you rotate by an exact fraction of the circle: the vectors distribute evenly, forming a regular polygon. The polygon is perfectly symmetric, so the sum of its vertices is 0.
This cancellation is the geometric meaning of *orthogonality*. It ensures that each frequency component is independent of the others, just like perpendicular directions in Euclidean space.

2.4 Fourier matrix and the DFT

We can arrange the complex exponentials into a matrix called the *Fourier matrix*:

$$F_N(i,j) \;=\; \omega_N^{ij}, \qquad i,j = 0,\dots,N-1,$$

where $\omega_N = e^{-2\pi i/N}$ is the primitive N-th root of unity.

The Fourier matrix explicitly. For $N = 4$, the primitive root of unity is

$$\omega_4 = e^{-2\pi i/4} = e^{-i\pi/2} = -i.$$

The Fourier matrix is

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & \omega_4 & \omega_4^2 & \omega_4^3 \\ 1 & \omega_4^2 & \omega_4^4 & \omega_4^6 \\ 1 & \omega_4^3 & \omega_4^6 & \omega_4^9 \end{bmatrix}.$$

Simplifying the powers of ω_4 gives

$$F_4 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 1 & -i & -1 & i \\ 1 & -1 & 1 & -1 \\ 1 & i & -1 & -i \end{bmatrix}.$$

In general, for arbitrary N, the Fourier matrix is

$$F_N = \begin{bmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega_N & \omega_N^2 & \cdots & \omega_N^{N-1} \\ 1 & \omega_N^2 & \omega_N^4 & \cdots & \omega_N^{2(N-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_N^{N-1} & \omega_N^{2(N-1)} & \cdots & \omega_N^{(N-1)(N-1)} \end{bmatrix}.$$

Note

Columns as a basis of $\mathbb{C}^N$.
Each column of F_N is a vector

$$v_k = \left(1, \omega_N^k, \omega_N^{2k}, \ldots, \omega_N^{(N-1)k}\right)^T, \qquad k = 0, 1, \ldots, N-1.$$

- These are the sampled exponentials $e^{2\pi i k t_j}$ on the uniform grid $t_j = j/N$. - The orthogonality of roots of unity means that $\{v_0, \ldots, v_{N-1}\}$ are mutually orthogonal vectors in $\mathbb{C}^N$. - After scaling by $1/\sqrt{N}$, they form an *orthonormal basis*.

So the DFT is nothing more than a *change of basis* from the standard basis to the Fourier basis.

What does this mean? - Each column of F_N corresponds to one frequency k. - Each row corresponds to one sample $t_j = j/N$. - The entry ω_N^{ij} is exactly $e^{-2\pi i k t_j}$: the value of frequency k at sample j.

So F_N is like a big "lookup table" of how every frequency looks on the sampling grid.

Forward DFT. Multiplying by F_N projects the data onto these frequencies:

$$c = F_N\, y,$$

where - $y = (y_0, \ldots, y_{N-1})^T$ are the samples, - $c = (c_0, \ldots, c_{N-1})^T$ are the Fourier coefficients (how much of each frequency is present).

Inverse DFT. Because the basis waves are orthogonal, the process is invertible. The inverse is

$$y = \frac{1}{N} F_N^H c,$$

where F_N^H is the conjugate transpose of F_N.

This means we can go back from frequencies to the original samples without losing information.

Note

Big picture. The DFT is just a change of coordinates:

- In the *time domain* the signal is described by its samples y_j at different times.
- In the *frequency domain* the same signal is described by coefficients c_k attached to different pure frequencies.

Because the Fourier basis functions are orthogonal (they don't interfere), this change of coordinates is stable and invertible. It is exactly like switching from Cartesian to polar coordinates in geometry: the same point, just described in another basis.

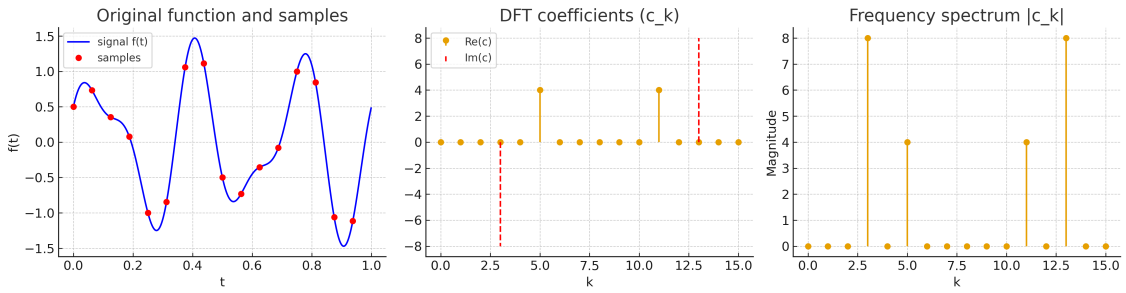

Figure 2: Pipeline: periodic function (top-left) → uniform samples y_j → DFT coefficients c_k (top-right) → frequency-domain magnitudes $|c_k|$ (bottom).

2.5 Properties and impact

- **Energy preservation.** The Fourier matrix is unitary up to scaling, so

$$\sum_{j=0}^{N-1} |y_j|^2 = \frac{1}{N} \sum_{k=0}^{N-1} |c_k|^2.$$

  This is the discrete analogue of Parseval's theorem.

- **Computation: FFT.** A naive DFT costs $O(N^2)$ operations. The *Fast Fourier Transform (FFT)* is a clever algorithm that reduces this to $O(N \log N)$. This efficiency is what makes real-time audio, video, and scientific computing possible.

2.6 Why is the DFT useful?

The Discrete Fourier Transform is not only a piece of linear algebra. It is the mathematical backbone of almost every digital technology: audio, images, video, communication, even medical scans.

Example: digital audio. When you speak into your phone:

1. The microphone converts sound waves into a continuous electrical signal $f(t)$.
2. The signal is **sampled** at a high rate (e.g. 8–16 kHz for phone calls, 44.1 kHz for music).
3. The DFT is applied to short windows of these samples to measure how much energy each frequency contains.
4. The frequency information can be:
   - stored (MP3, AAC compression keeps only the important frequencies for the ear),
   - transmitted (cellular networks send compact frequency-domain data),
   - or manipulated (noise cancellation, equalizers).
5. On playback, the inverse DFT reconstructs the sound wave from its frequency representation.

Why do we sample? A real sound wave $f(t)$ is a continuous function of time. Even in just *one second* of recording, t takes uncountably many values. Storing all of them would require infinite memory.

Instead, digital systems use two key ideas:

1. **Sampling:** We only keep values of $f(t)$ at equally spaced times $t_j = j/N$. For audio, N is large but finite (e.g. 44,100 samples per second for CD quality).
2. **Fourier analysis:** We apply the DFT to those samples. This transforms the time sequence y_j into a frequency representation c_k. Instead of storing every wiggle of the waveform, we just store how much of each sine/cosine wave is present.

Note

Big picture. Continuous audio cannot be stored directly — it would require infinitely many data points even for one second. By combining *sampling* (make it finite) with the *DFT* (make it interpretable as frequencies), we obtain a practical representation. This is why your phone can record, compress, transmit, and replay sound in real time.

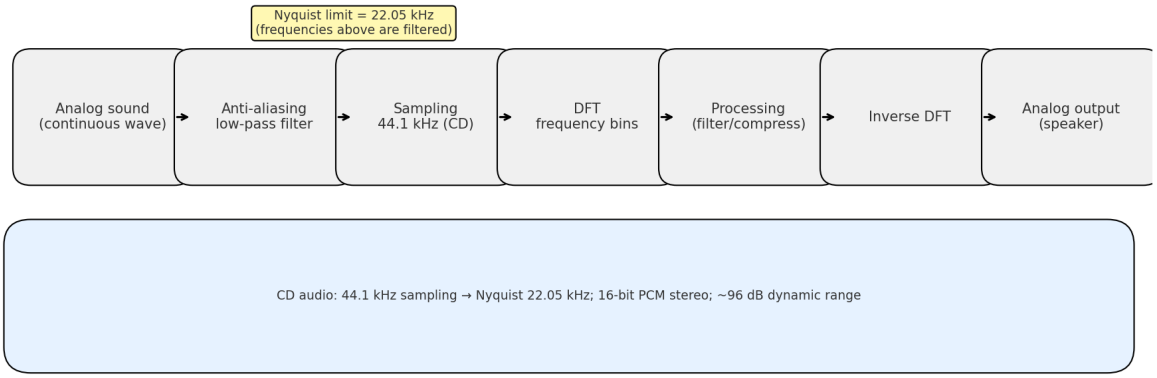

Figure 3: Audio pipeline: analog sound is sampled (44.1 kHz for CD), transformed via DFT into frequency bins, processed (filtering/compression), then reconstructed back to analog output. Nyquist limit at 22.05 kHz ensures no aliasing in the audible range.

3 DFT in Python: From Theory to Computation

After understanding the mathematical structure of the Discrete Fourier Transform, a natural question arises:

> *What would it look like if we tried to implement the DFT directly in Python?*

We have just seen that the DFT transforms a vector of samples

$$y = (y_0, y_1, \ldots, y_{N-1})$$

into frequency coefficients

$$c_k = \sum_{j=0}^{N-1} y_j \, e^{-2\pi i j k/N}, \qquad k = 0, \ldots, N-1.$$

Mathematically elegant — but computationally expensive.

Let's reason about what this means in practice.

Naive implementation. A straightforward implementation would literally follow the formula: for each k, compute the sum over all N samples y_j multiplied by complex exponentials. In pseudocode:

```
for k in range(N):
    c[k] = 0
    for j in range(N):
        c[k] += y[j] * exp(-2i * j * k / N)
```

This involves N operations inside another loop of length N, leading to a total of N^2 multiplications and additions. That is, the cost grows quadratically with the number of samples.

Note

Complexity. If $N = 1024$, this means more than one million operations for just one DFT. If $N = 1{,}000{,}000$, the number of operations explodes to 10^{12}, which is far beyond what real-time applications could afford.

Is this sustainable? Clearly, no — at least not for modern data sizes. Every second of CD-quality audio contains 44,100 samples. Computing an $O(N^2)$ transform for each second of sound would make real-time audio impossible.

This motivates a new question:

> *Can we exploit the internal structure of the Fourier matrix to compute the same result faster?*

The answer is yes — and this is the central idea behind the **Fast Fourier Transform (FFT)**, a family of algorithms that reduce the computational cost from $O(N^2)$ to $O(N \log N)$ without

changing the result.

Note

Historical importance. The FFT was not just a mathematical trick — it was a revolution. Introduced in efficient form by Cooley and Tukey in 1965, it made it feasible to perform real-time spectral analysis, digital communication, and image processing. Without it, technologies such as mobile phones, MP3, or MRI imaging would have been practically impossible.

What happens in Python? In Python (and in most modern languages), we rarely compute the DFT manually. Libraries like `numpy.fft` implement the FFT algorithm internally. This means that when we call

```
import numpy as np

# Forward and inverse Fourier Transform
c = np.fft.fft(y)
y_rec = np.fft.ifft(c)
```

we are not using the naive $O(N^2)$ version, but the optimized $O(N \log N)$ FFT. The first line computes the forward transform (from time domain to frequency domain), and the second one performs the inverse transform (back to time domain).

Note

Bridge to applications. These two functions — `fft` and `ifft` — are the computational bridge between the theoretical DFT we derived and the practical signal processing systems that power the modern world. Every audio filter, image compression, or vibration analysis starts with exactly this pair of transforms.

4 The Fast Fourier Transform (FFT)

4.1 Motivation

Up to this point, we have seen that the Discrete Fourier Transform (DFT) converts samples y_j into frequency coefficients c_k. Mathematically it is simple:

$$c_k = \sum_{j=0}^{N-1} y_j \, e^{-2\pi i j k/N}.$$

But this definition hides a problem.

The problem: cost. Computing each c_k requires a sum over all N samples. Since there are N different k's, we need $N \times N = N^2$ multiplications. That means:

$$\text{Cost of the DFT: } O(N^2).$$

For $N = 1{,}000$, this is already one million operations. For $N = 1{,}000{,}000$, it becomes one trillion. This was completely impractical before computers became fast.

A natural question. *Can we compute the same result faster, without changing the meaning?*

That question led to one of the greatest discoveries in numerical science: the **Fast Fourier Transform (FFT)**.

> Note
>
> **Big picture.** The FFT is not a new transform — it gives the *same* result as the DFT. What changes is *how* we compute it: by reorganizing the same arithmetic in a smarter way.

4.2 The core idea: divide and conquer

The DFT formula looks like one big sum, but there is hidden structure inside. If N is even, we can split the samples into two halves: the **even-indexed** samples and the **odd-indexed** samples.

Let

$$y_j^{(even)} = y_{2j}, \qquad y_j^{(odd)} = y_{2j+1}, \qquad j = 0, \ldots, N/2 - 1.$$

The Discrete Fourier Transform is defined as

$$c_k = \sum_{n=0}^{N-1} y_n \, e^{-2\pi i n k/N}.$$

Then the DFT can be rewritten as

$$c_k = \sum_{j=0}^{N/2-1} y_{2j} e^{-2\pi i(2j)k/N} + e^{-2\pi ik/N} \sum_{j=0}^{N/2-1} y_{2j+1} e^{-2\pi i(2j)k/N}.$$

Notice that both sums have the same exponential term $e^{-2\pi i(2j)k/N}$, which is just the DFT of size $N/2$.

This means: we can compute two smaller DFTs of size $N/2$, and then combine their results with a few extra multiplications. That reduces the total amount of work dramatically.

$$T(N) = 2T(N/2) + O(N) \quad \Rightarrow \quad T(N) = O(N \log N).$$

Note

Intuition. Instead of doing all N^2 multiplications directly, we reuse computations by splitting the problem into halves. At each level we only pay $O(N)$ work to merge the results. The FFT is a perfect example of the *divide and conquer* principle.

4.3 A small example ($N = 8$)

To see how the pattern works, consider $N = 8$. We first split the signal into even and odd samples:

$$(y_0, y_2, y_4, y_6), \qquad (y_1, y_3, y_5, y_7).$$

We compute two DFTs of length 4 (each costs much less), and then combine them using the so-called **twiddle factors**

$$\omega_8^k = e^{-2\pi ik/8}.$$

The recombination step is:

$$c_k = c_k^{(even)} + \omega_8^k \, c_k^{(odd)}, \qquad c_{k+4} = c_k^{(even)} - \omega_8^k \, c_k^{(odd)}, \quad k = 0, 1, 2, 3.$$

So from two small transforms we build one larger transform. Then the same trick applies recursively, until we reach size $N = 2$, where the DFT is trivial.

Note

Recursive structure. Each FFT step splits the signal in half and recombines the results. After $\log_2 N$ levels, we have processed all data. That is where the $O(N \log N)$ cost comes from.

5 Frequency Filtering via the DFT

So far we have learned how the Discrete Fourier Transform (DFT) decomposes a signal into pure frequencies. Now we explore one of the most powerful consequences of this idea: **we can modify a signal by acting directly on its frequency components.**

5.1 From time to frequency and back

The DFT transforms a sequence of samples $y = (y_0, y_1, \ldots, y_{N-1})$ into its frequency representation $c = (c_0, c_1, \ldots, c_{N-1})$. Each coefficient c_k tells us how much of frequency k is present in the signal.

The inverse transform reconstructs the original signal by combining these frequencies:

$$y_j = \frac{1}{N} \sum_{k=0}^{N-1} c_k \, e^{2\pi i j k/N}.$$

Once we are in the frequency domain, we can *modify* the coefficients before transforming back. This process is known as **frequency filtering**.

Conceptual pipeline.

$$y \xrightarrow{\text{DFT}} c \xrightarrow{\text{Filtering}} c' \xrightarrow{\text{Inverse DFT}} y'$$

- y – the original discrete signal (time domain);
- c – the DFT coefficients (frequency domain);
- c' – the modified coefficients after filtering;
- y' – the reconstructed, filtered signal.

Note

Intuitive idea. In the time domain, filtering looks complicated — it involves convolutions and averages. In the frequency domain, filtering is simple: we just choose which frequencies to keep or remove. Each coefficient c_k acts like a "volume knob" for a specific tone in the signal.

5.2 Types of frequency filters

Every real signal contains a mix of slow and fast variations:

- slow, smooth trends (low frequencies);
- rapid oscillations, sharp transitions, or noise (high frequencies).

By manipulating different frequency ranges, we can shape how the signal behaves.

1. Low-pass filter. This type of filter keeps only the slow oscillations (the low frequencies) and removes the fast ones. Mathematically:

$$c'_k = \begin{cases} c_k, & |k| \le k_c, \\ 0, & |k| > k_c, \end{cases}$$

where k_c is the cutoff frequency.

- Removes rapid fluctuations and high-frequency noise.
- Produces a smooth, "denoised" version of the signal.
- Used in audio smoothing, image blurring, and data compression.

2. High-pass filter. The opposite idea: remove the slow variations and keep only the fast oscillations.

$$c'_k = \begin{cases} 0, & |k| \le k_c, \\ c_k, & |k| > k_c. \end{cases}$$

- Emphasizes edges, abrupt changes, or fine details.
- Used in image sharpening, feature extraction, and motion detection.

3. Band filters. Sometimes we want to isolate only a specific range of frequencies:

- **Band-pass:** keeps frequencies between k_1 and k_2.
- **Band-stop:** removes only those frequencies, keeping all others.

Note

Physical meaning. Low frequencies describe slow variations (the overall shape of a melody or an image). High frequencies describe fast details (sharp edges or background hiss). Filtering lets us decide which part of the signal we care about.

5.3 Example: Filtering a signal (conceptual view)

Imagine an audio signal $y(t)$ sampled at 8 kHz (so $N = 8000$ samples per second). After applying the DFT, we obtain c_k coefficients that tell us how much energy lies in each frequency.

- Suppose the useful sound is below 1000 Hz. - Frequencies above that are likely just background noise.

To clean the sound:

$$c'_k = \begin{cases} c_k, & |f_k| \le 1000 \text{ Hz}, \\ 0, & \text{otherwise}. \end{cases}$$

Then we compute the inverse DFT of c' to obtain $y'(t)$ — the filtered signal. All high-frequency hiss disappears, leaving a smoother, clearer sound.

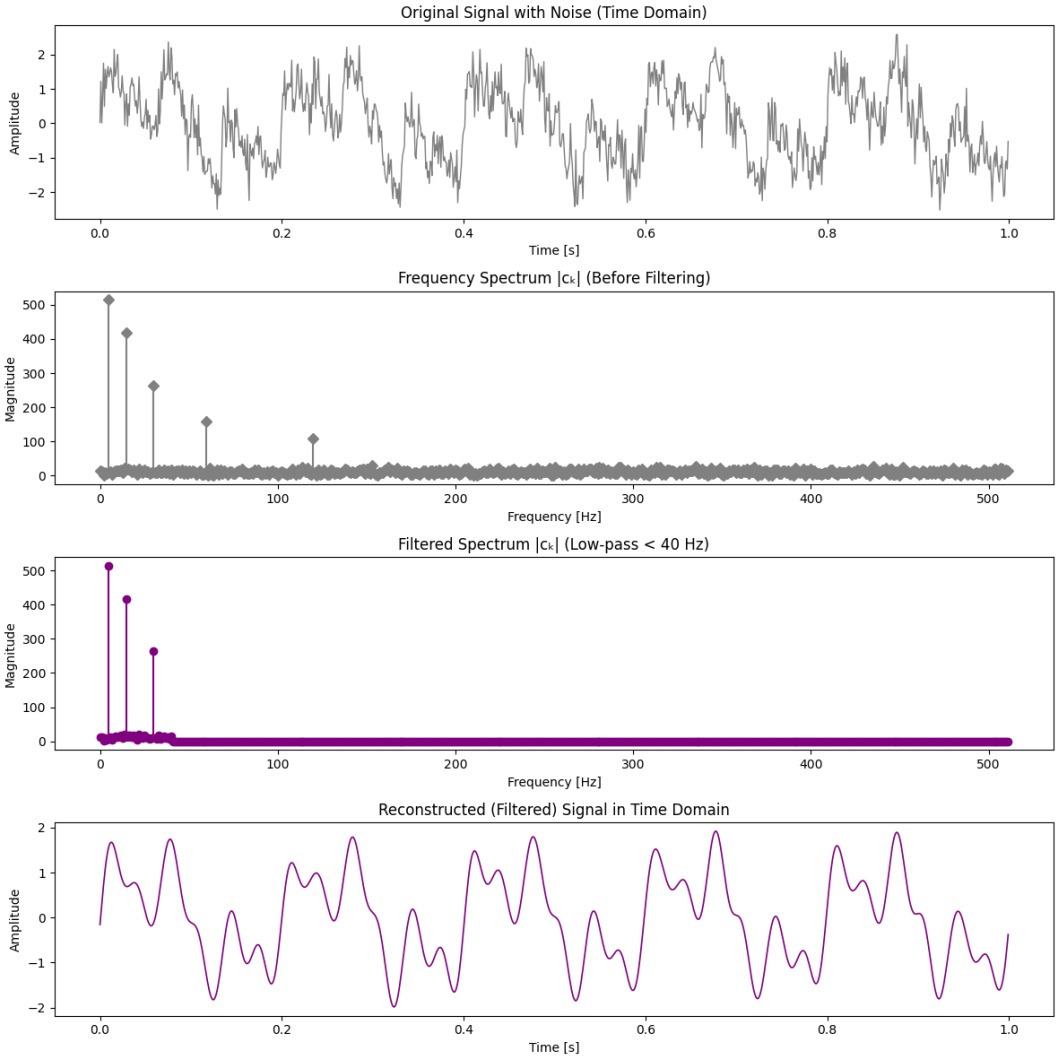

Figure 4: Frequency-domain filtering pipeline (N = 1024). Top to bottom: original noisy signal, magnitude spectrum before filtering, filtered spectrum after applying a low-pass mask, and reconstructed smooth signal. Filtering in frequency space is equivalent to convolution with a smoothing kernel in time.

5.4 Implementation intuition (in Python)

Using NumPy, frequency-domain filtering is conceptually straightforward:

```
import numpy as np

# Forward transform (time → frequency)
c = np.fft.fft(y)

# Design a simple low-pass filter
cutoff = N // 8
mask = np.zeros(N)
mask[:cutoff] = 1
```

```
mask[-cutoff:] = 1

# Apply filter in frequency domain
c_filtered = c * mask

# Inverse transform (frequency → time)
y_filtered = np.fft.ifft(c_filtered)
```

Note

Observation. In this short code, the key step is not the FFT itself — it's the filter mask. Multiplying c by the mask zeroes out unwanted frequencies, and the inverse transform reconstructs the filtered signal. This shows how directly DFT coefficients control the signal's behavior.

Note

Observation on DFT symmetry. For real signals, the DFT spectrum is symmetric: the second half ($k > N/2$) mirrors the first. This happens because frequencies above Nyquist "wrap around" the circle of complex exponentials. In practice, only the first half of the coefficients carries unique information.

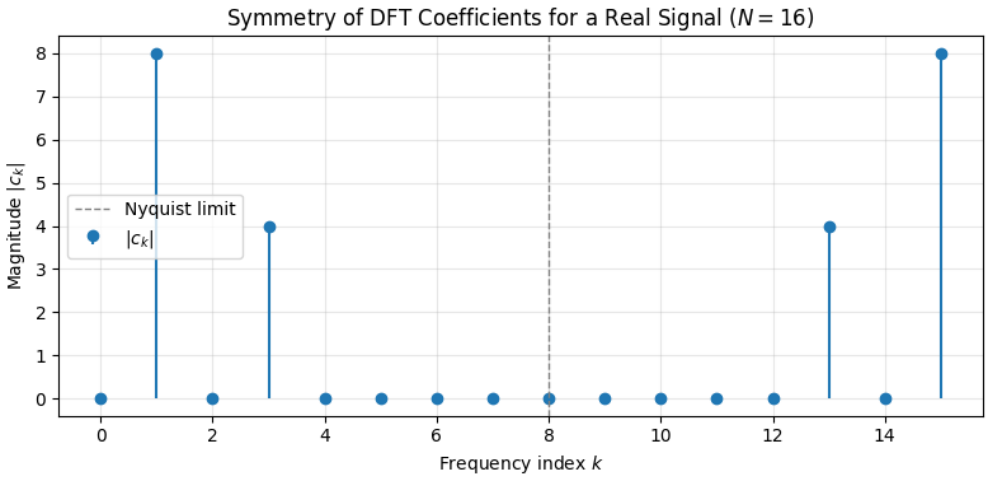

Figure 5: DFT coefficients for $N = 16$. Frequencies beyond $k = N/2$ repeat the same information in reverse order. This symmetry motivates the use of `fftshift`, which centers the zero frequency for clearer plots.

5.5 Visualizing frequencies: `fftshift`

By default, the output of `np.fft.fft` orders frequencies as:

$$[0, 1, 2, \ldots, N/2 - 1, -N/2, \ldots, -1](Figure 5).$$

This is correct mathematically, but confusing visually: the low frequencies (near 0) sit at the beginning and end of the array, while the high ones are split in the middle.

To make the spectrum easier to interpret, we use:

`np.fft.fftshift(c)`.

This simply *rearranges* the coefficients so that: - the zero frequency is at the center, - negative frequencies appear on the left, - positive frequencies on the right.

```
import numpy as np
import matplotlib.pyplot as plt

c = np.fft.fft(y)
c_shifted = np.fft.fftshift(c)
freqs = np.fft.fftshift(np.fft.fftfreq(N, d=1/N))

plt.plot(freqs, np.abs(c_shifted))
plt.title("Centered frequency spectrum via fftshift")
plt.xlabel("Frequency (Hz)")
plt.ylabel("|c_k|")
plt.show()
```

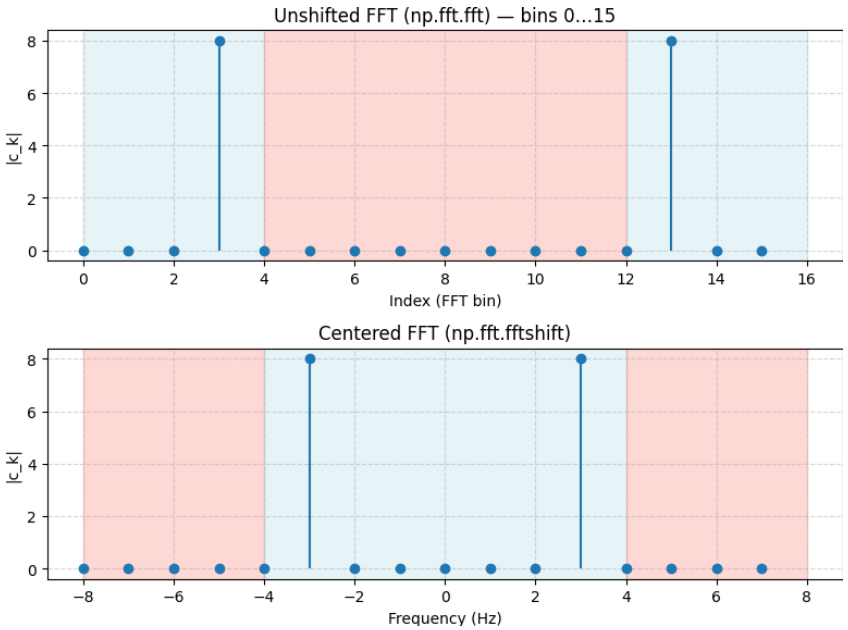

Figure 6: Comparison between the unshifted spectrum (`np.fft.fft`) and the centered one (`np.fft.fftshift`). In the top plot, FFT bins go from 0 to 15, while in the bottom plot the frequencies are rearranged so that the zero frequency is in the center, with negative frequencies on the left and positive on the right.

Note

Geometric interpretation. Frequencies come in positive and negative pairs: k and $N - k$ correspond to waves rotating in opposite directions. `fftshift` simply reorders them symmetrically around zero, so that the entire spectrum can be seen at once. This symmetry also reflects the fact that real signals have complex-conjugate symmetric spectra.

Circular reordering via `fftshift`. After applying `np.fft.fftshift`, the FFT coefficients are *circularly shifted* by half the array length. In practice, this means that the second half of the array moves to the front, and the first half moves to the end. For example, the original indices

$$[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15]$$

become, after applying `fftshift`,

$$[8, 9, 10, 11, 12, 13, 14, 15, 0, 1, 2, 3, 4, 5, 6, 7].$$

This rearrangement centers the zero-frequency component in the spectrum, placing negative frequencies on the left and positive ones on the right.

5.6 Why filtering matters

Filtering is not just a mathematical exercise — it's a fundamental tool in modern technology.

- **Audio:** removing hiss, equalizing bass/treble, compressing data.
- **Images:** sharpening, edge detection, noise removal, blurring.
- **Communication:** isolating desired frequency bands, avoiding interference.
- **Medicine:** extracting clean ECG or EEG signals from noisy data.

Note

Big picture. The DFT allows us to *see*, *understand*, and *control* the frequency content of data. It turns raw time samples into a structured view of information. Filtering is how we use that view to shape signals — the bridge between theory and technology.

6 Circulant matrices: the algebra behind the FFT

6.1 Motivation: why circulant matrices appear naturally

It is quite difficult to appreciate the role of circulant matrices if you have never discussed convolutions — especially *periodic* convolutions. For those who are curious and want to explore this connection in more detail, I will attach a few optional video links below (from previous years' C++ lectures):

Additional resources:

- 4.1.1 Discrete Finite Linear Time-Invariant Causal Channels/Filters (10.46 min)
- 4.1.2 LT-FIR Linear Mappings (11.50 min)
- 4.1.3 Discrete Convolutions (8.37 min)
- 4.1.4 Periodic Convolutions (12.07 min)
- 4.2.1 Diagonalizing Circulant Matrices (16.32 min)
- 4.2.2 Discrete Convolution via Discrete Fourier Transform (6.54 min)

But even without knowing much about convolutions, let's think through a simple and concrete example to see how important circulant matrices really are.

Suppose we have a line of N discrete values

$$y_0, y_1, \dots, y_{N-1},$$

and we want to smooth them — that is, replace each value by the average of itself and its two neighbors:

$$\tilde{y}_j = \frac{1}{3}(y_{j-1} + y_j + y_{j+1}).$$

To make this operation *periodic*, we wrap around the ends, so that $y_{-1} = y_{N-1}$ and $y_N = y_0$.

Now the key question: *How could we express this averaging operation using a matrix?*

The answer is — a **circulant matrix**.

$$\mathrm{Circ}(c) = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & 0 & 0 & \cdots & 0 & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & \cdots & 0 & 0 \\ 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & \cdots & 0 & 0 \\ \vdots & & \ddots & \ddots & \ddots & & \vdots \\ 0 & 0 & \cdots & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 \\ \frac{1}{3} & 0 & \cdots & 0 & 0 & \frac{1}{3} & \frac{1}{3} \end{bmatrix}.$$

Each row is simply a one-step shift of the previous one. This circular structure captures exactly the periodic nature of our averaging rule.

And here comes the surprising fact: **no matter what circulant matrix you choose, its eigenvectors are precisely the Fourier vectors.**

That is, the Fourier basis diagonalizes every circulant matrix — a deep algebraic connection that forms the heart of the FFT.

Note

In short: Every circulant matrix represents a periodic linear operation (a circular convolution). The Fourier transform provides the coordinates where such operations become diagonal — each frequency component evolves independently.

6.2 Definition and notation

Let $c = (c_0, c_1, \ldots, c_{N-1})^\top \in \mathbb{C}^N$. The *circulant matrix* generated by c is

$$\mathrm{Circ}(c) = \begin{bmatrix} c_0 & c_{N-1} & c_{N-2} & \cdots & c_1 \\ c_1 & c_0 & c_{N-1} & \cdots & c_2 \\ c_2 & c_1 & c_0 & \cdots & c_3 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ c_{N-1} & c_{N-2} & c_{N-3} & \cdots & c_0 \end{bmatrix}, \qquad (\mathrm{Circ}(c))_{ij} = c_{(i-j) \bmod N}.$$

Multiplying $y = \mathrm{Circ}(c)\, x$ is exactly the *circular convolution* (mod N) of c and x.

Visual example (N=5). Let $x = (x_0, x_1, x_2, x_3, x_4)^\top$ and let the averaging filter be $c = (\frac{1}{3}, \frac{1}{3}, 0, 0, \frac{1}{3})^\top$. Then the circulant matrix is:

$$\mathrm{Circ}(c) = \begin{bmatrix} \frac{1}{3} & \frac{1}{3} & 0 & 0 & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 & 0 \\ 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} & 0 \\ 0 & 0 & \frac{1}{3} & \frac{1}{3} & \frac{1}{3} \\ \frac{1}{3} & 0 & 0 & \frac{1}{3} & \frac{1}{3} \end{bmatrix}.$$

Multiplying $y = \mathrm{Circ}(c)\, x$ gives:

$$\begin{aligned} y_0 &= \tfrac{1}{3}(x_0 + x_1 + x_4), \\ y_1 &= \tfrac{1}{3}(x_0 + x_1 + x_2), \\ y_2 &= \tfrac{1}{3}(x_1 + x_2 + x_3), \\ y_3 &= \tfrac{1}{3}(x_2 + x_3 + x_4), \\ y_4 &= \tfrac{1}{3}(x_3 + x_4 + x_0). \end{aligned}$$

Each output y_j is the average of x_j and its two neighbors, with the indices wrapping around at the ends. That "wrap-around" is what makes the operation *circular*.

Visually:

$$\begin{array}{ccccc} x_4 & \to & x_0 & \to & x_1 \\ & & \downarrow & & \downarrow \\ & & y_0 & \leftarrow & y_1 & \leftarrow \cdots \end{array}$$

The structure of the matrix makes this periodic neighborhood relation explicit: every row is just a shifted version of the first one.

Note

Periodic shift. Define the shift matrix

$$S = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \end{bmatrix}, \qquad (Sx)_j = x_{(j-1) \bmod N}.$$

Then every circulant can be written as

$$\mathrm{Circ}(c) = \sum_{m=0}^{N-1} c_m \, S^m.$$

This immediately implies that if we know the eigenpairs of S, we also know those of any circulant matrix.

We can verify this claim directly.

First, note that for the shift matrix

$$S = \begin{bmatrix} 0 & 1 & 0 & \cdots & 0 \\ 0 & 0 & 1 & \cdots & 0 \\ \vdots & & & \ddots & \vdots \\ 0 & 0 & 0 & \cdots & 1 \\ 1 & 0 & 0 & \cdots & 0 \end{bmatrix}, \qquad (Sx)_j = x_{(j-1) \bmod N},$$

applying S once shifts all elements of x one position down (with wrap-around). If we apply S twice, we get a two-step shift:

$$(S^2 x)_j = x_{(j-2) \bmod N},$$

and in general, by simple induction,

$$(S^m x)_j = x_{(j-m) \bmod N}.$$

This means that S^m performs a circular shift by m positions.

Now consider a linear combination of these shifted versions:

$$C = \sum_{m=0}^{N-1} c_m \, S^m.$$

The j-th entry of Cx is then

$$(Cx)_j = \sum_{m=0}^{N-1} c_m \, (S^m x)_j = \sum_{m=0}^{N-1} c_m \, x_{(j-m) \bmod N}.$$

But this is exactly the definition of a *circular convolution* between c and x. Hence C is the circulant matrix generated by c, and each entry satisfies

$$C_{ij} = c_{(i-j) \bmod N}.$$

This shows that every circulant matrix can be written as a polynomial in the shift matrix S.

6.3 Short recap: eigenvalues and eigenvectors

Let $A \in \mathbb{C}^{N \times N}$ be a square matrix. A scalar $\lambda \in \mathbb{C}$ is called an *eigenvalue* of A if there exists a nonzero vector v (called an *eigenvector*) such that

$$Av = \lambda v.$$

In words, multiplying v by A only stretches or shrinks it by a factor λ, but does not change its direction.

If the matrix A has N linearly independent eigenvectors, we can collect them as the columns of a matrix

$$V = [v_1 \mid v_2 \mid \cdots \mid v_N],$$

and the corresponding eigenvalues on the diagonal of

$$\Lambda = \operatorname{diag}(\lambda_1, \ldots, \lambda_N).$$

Then the action of A can be written as

$$A = V \, \Lambda \, V^{-1}.$$

This is called the **diagonalization** of A.

However, not every matrix can be diagonalized in this way using a simple inverse. Things become much simpler if A is what we call a *normal* matrix, that is, if it satisfies

$$AA^H = A^H A,$$

where A^H is the conjugate transpose of A.

For normal matrices, the following important result holds:

Spectral Theorem (for normal matrices). Every normal matrix A can be diagonalized by a unitary matrix U:

$$A = U\,\Lambda\,U^H, \qquad U^H U = I.$$

Here:

- The columns of U are the eigenvectors of A, and they are *orthonormal* ($U^H U = I$).
- The diagonal entries of Λ are the eigenvalues of A.

Why this is useful. Unitary matrices preserve lengths and angles, so rewriting A in this way is numerically stable and easy to interpret: A simply scales each eigenvector u_k by its eigenvalue λ_k:

$$Au_k = \lambda_k u_k.$$

This property is the reason we can safely use such diagonal forms later when studying the structure of circulant matrices.

6.4 Diagonalization of the shift matrix (and all circulants)

Let $\omega_N = e^{-2\pi i/N}$. For $k = 0, \ldots, N-1$, define the vectors

$$v^{(k)} = \begin{bmatrix} 1 & \omega_N^k & \omega_N^{2k} & \cdots & \omega_N^{(N-1)k} \end{bmatrix}^\top.$$

A direct calculation shows that

$$S\,v^{(k)} = \omega_N^k\,v^{(k)}.$$

Hence, each $v^{(k)}$ is an eigenvector of S with eigenvalue ω_N^k.

The matrix that contains them as columns

$$V = [\,v^{(0)}|v^{(1)}|\cdots|v^{(N-1)}\,]$$

is exactly the same as the (unnormalized) **DFT matrix**. From what we know about the DFT, this matrix is invertible — hence its columns (the $v^{(k)}$) must be linearly independent.

If you look again at the DFT formula, you will notice that these vectors are not only independent, but also *orthogonal* to one another — their inner products vanish when $k \neq \ell$. This means we can even normalize them to obtain a unitary matrix

$$U = \frac{1}{\sqrt{N}} V, \qquad U^H U = I.$$

Because we now have N independent eigenvectors, we have fully diagonalized the shift matrix S:

$$S = V\,\mathrm{diag}(\omega_N^0, \omega_N^1, \ldots, \omega_N^{N-1})\,V^{-1}.$$

(If you want to confirm this visually, have a look at the DFT again — its basis vectors are exactly these $v^{(k)}$.)

Therefore:

- all circulant matrices share the same eigenvectors $v^{(k)}$;
- their eigenvalues are

$$\lambda_k = \sum_{m=0}^{N-1} c_m \, \omega_N^{mk}, \qquad k = 0, \ldots, N-1.$$

With $U = \frac{1}{\sqrt{N}} V$, this gives the unitary diagonalization

$$C = U \, \mathrm{diag}(\lambda_0, \ldots, \lambda_{N-1}) \, U^H.$$

6.5 Small example (N=4)

Let $c = (c_0, c_1, c_2, c_3)^\top$ and $\omega_4 = e^{-2\pi i/4} = -i$. Then the eigenvalues are:

$$\begin{aligned}
\lambda_0 &= c_0 + c_1 + c_2 + c_3, \\
\lambda_1 &= c_0 + c_1(-i) + c_2(-1) + c_3(i), \\
\lambda_2 &= c_0 + c_1(-1) + c_2(1) + c_3(-1), \\
\lambda_3 &= c_0 + c_1(i) + c_2(-1) + c_3(-i).
\end{aligned}$$

The eigenvectors (unnormalized) are $v^{(k)} = (1, \omega_4^k, \omega_4^{2k}, \omega_4^{3k})^\top$. In this basis, $\mathrm{Circ}(c)$ becomes diagonal with the eigenvalues above.